03 · Duplicate Protection

Layered defences against doing the same thing twice, and an honest map of where duplicates still get through.

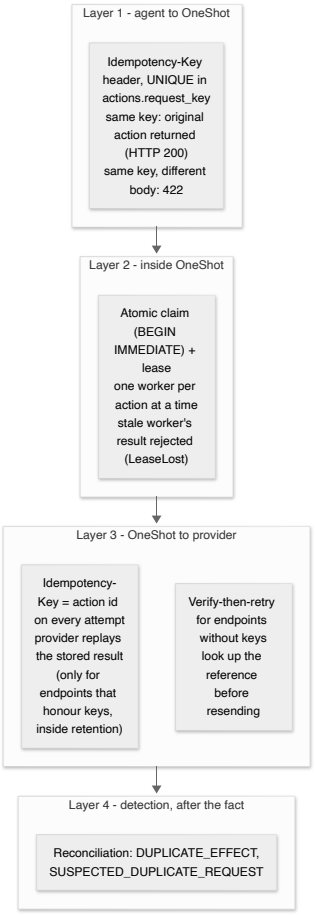
The problem

Duplicates come from retries, and retries come from uncertainty. There are three places where a request can be repeated:

1. **The agent → OneShot**: the agent's HTTP call to OneShot timed out, so the agent sends it again.
2. **Inside OneShot**: two workers pick up the same action, or a recovered worker repeats an attempt.
3. **OneShot → the provider**: OneShot retries after an ambiguous outcome.

No single mechanism covers all three. OneShot stacks one defence per layer and adds *detection* for whatever gets past them.

The layers



Layer 1: agent → OneShot (`oneshot/api.py` , `ActionStore.submit`)

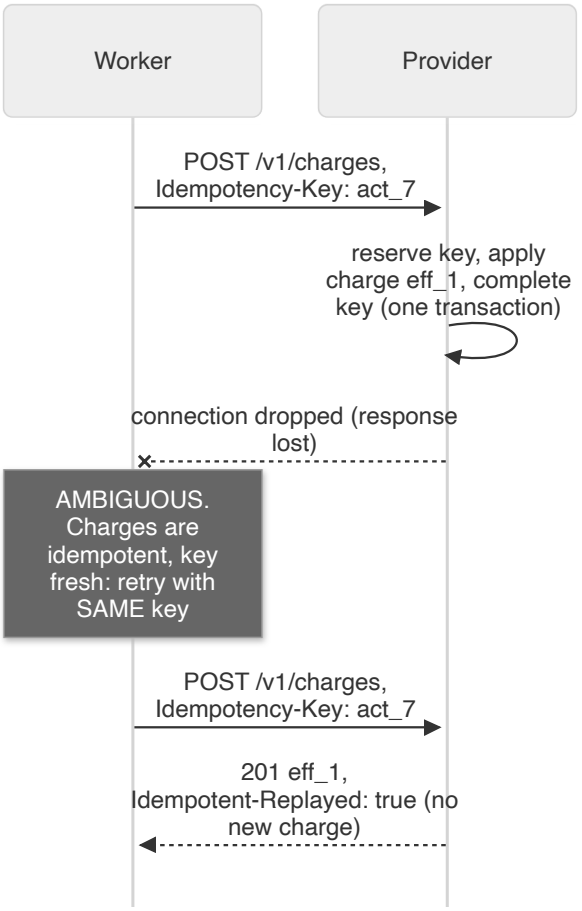
`POST /actions` *requires* an `Idempotency-Key` . The store keeps it in a UNIQUE column and returns the existing action for a repeat. The check-then-insert runs inside `BEGIN IMMEDIATE` , so even concurrent duplicates from several API processes create exactly one action (`test_concurrent_duplicate_submissions_create_one_action` runs 8 at once). The status codes follow the IETF draft *The Idempotency-Key HTTP Header Field*: 201 new, 200 + `Idempotent-Replayed: true` for a replay, 422 for reuse with a different payload.

Layer 2: inside OneShot (`ActionStore.claim_next` , `apply_decision`)

Only one worker can claim a due action; claiming is an atomic UPDATE under the write lock. If a worker stalls beyond its lease and another takes over, the stalled worker's late result is rejected (`LeaseLost`) instead of overwriting the newer decision.

Layer 3: OneShot → provider (`ProviderClient.send` , `policy.after_ambiguous`)

The provider idempotency key is the **action id**. It is created when the action is accepted and persisted, so it is identical across every retry, every worker and every crash. For `payment.charge` , the provider's endpoint honours keys:



For `message.send`, the endpoint ignores keys (many real SMS and email APIs behave like this), so OneShot verifies before resending (doc 02).

Layer 4: detection (`oneshot/reconciler.py`)

Whatever slips through is detected by comparing OneShot's records with the provider's ledger: more than one effect for one action (`DUPLICATE_EFFECT`, critical), or separate actions with identical content created close together (`SUSPECTED_DUPLICATE_REQUEST`, a warning only, because two identical coffees can be intentional).

Where duplicates still slip through — honestly

#	Gap	Why no layer stops it	Shown by
1	The agent retries with a **new** Idempotency-Key	To OneShot a new key is a new intent. Content-based blocking would also block legitimate repeats.	demo `agent-duplicates`; `test_agent_retrying_with_new_key_creates_duplicate_that_reconciliation_flags`
2	Non-idempotent provider **slower than the settle window**	Verification sees "absent", OneShot resends, then the original lands.	demo `slow-provider-duplicate`; `test_duplicate_slips_through_when_provider_is_slower_than_settle_window`

#	Gap	Why no layer stops it	Shown by
3	`provider_key_retention_seconds` configured **longer** than the provider's real retention	A "safe" same-key retry arrives after the provider forgot the key.	`test_misconfigured_retention_window_causes_a_duplicate`
4	A worker stalls past its lease while its request to a **non-idempotent** provider is still in flight	The worker that takes over verifies, may find nothing yet, and resend. Leases are not fencing tokens that the provider can check.	documented; mitigated by `lease_seconds > 2 × provider_timeout_seconds` (enforced in `Settings`)
5	OneShot's database is **lost or restored from an old backup**	The agent-key records are gone, so a resubmission looks new.	demo `database-wiped` shows the effects surface as `ORPHAN_EFFECT`

The honest summary is that **OneShot prevents duplicates when the provider deduplicates and the configuration is right, reduces them otherwise, and always makes the remaining ones visible.**

Observed: the provider deduplicates a same-key retry (demo)

```
$ python -m oneshot demo lost-response-charge
# captured 2026-11-03 by scripts/capture_outputs.py (exit code 0)

──────────────────────────────────────── OneShot demo ────────────────────────────────────────
Each scenario uses a fresh provider process with its own ledger, a fresh OneShot database, and real
worker processes. Faults are injected at the provider (or as failpoints in the worker).
Working directory: $TMPDIR/oneshot-demo-9z0cyuem

1/1  lost-response-charge: Charge applied, but the response is lost (idempotent endpoint) ──────────
  > fault armed at the provider: lost_response on /v1/charges
  > agent POST /actions Idempotency-Key=agent-order-1004 -> HTTP 201 act_37480b15f188: charge
cus_ada 42.00 USD
  > start worker process
  > worker process finished: every action is SUCCEEDED, FAILED or UNKNOWN (exit 0)
  OneShot's record (event timeline):
  act_37480b15f188  charge cus_ada 42.00 USD  SUCCEEDED
    +  0.00s (new) -> PENDING  accepted and durably recorded
    +  0.07s PENDING -> IN_FLIGHT  attempt 1: claimed by worker-87184, intent recorded before
sending
    +  0.07s IN_FLIGHT -> PENDING  connection dropped before a response arrived; outcome unknown ->
retry in 0.30s with the SAME idempotency key (provider deduplicates, so a retry cannot double-apply)
    +  0.41s PENDING -> IN_FLIGHT  attempt 2: claimed by worker-87184, intent recorded before
sending
    +  0.42s IN_FLIGHT -> SUCCEEDED  provider confirmed: replayed original result (provider
deduplicated) as eff_00001
  Provider's side of the story (its log):
    POST /v1/charges ref=act_37480b15f188 key=act_37480b15f188 mode=lost_response -> APPLIED
eff_00001, response deliberately dropped
    POST /v1/charges ref=act_37480b15f188 key=act_37480b15f188 -> REPLAY of stored response (no new
effect)
  Provider's independent ledger:
    act_37480b15f188: 1 effect(s) ['eff_00001']
  [PASS] act_37480b15f188 is SUCCEEDED and provider ledger has 1 effect(s) (observed: SUCCEEDED, 1)
  Lesson: OneShot cannot know whether the charge happened. Because the provider deduplicates by
Idempotency-Key, retrying with the SAME key is safe: the provider replays the original result.

Scorecard (every expectation checked against the
provider's real ledger)
┏━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━┳━━━━━━┳━━━━━━━━┓
┃ scenario           ┃ checks ┃ time ┃ result ┃
┡━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━╇━━━━━━╇━━━━━━━━┩
│ lost-response-charge │    1/1 │ 0.5s │ PASS   │
└────────────────────┴────────┴──────┴────────┘
```

Observed: agent-layer duplicates (demo)

```
$ python -m oneshot demo agent-duplicates
# captured 2026-11-03 by scripts/capture_outputs.py (exit code 0)

──────────────────────────────────────────── OneShot demo ────────────────────────────────────────────
Each scenario uses a fresh provider process with its own ledger, a fresh OneShot database, and real
worker processes. Faults are injected at the provider (or as failpoints in the worker).
Working directory: $TMPDIR/oneshot-demo-v87oe3rq

1/1  agent-duplicates: The agent submits the same request twice ──────────────────────────────────────
  > agent POST /actions Idempotency-Key=agent-order-1005 -> HTTP 201 act_096200983a97: charge
cus_ada 42.00 USD
  > agent POST /actions Idempotency-Key=agent-order-1005 -> HTTP 200 act_096200983a97: charge
cus_ada 42.00 USD (replayed: same key, no new action)
  [PASS] same key returned the same action
  > agent POST /actions Idempotency-Key=agent-order-1005-retry -> HTTP 201 act_e1e0de824cfe: charge
cus_ada 42.00 USD
  > start worker process
  > worker process finished: every action is SUCCEEDED, FAILED or UNKNOWN (exit 0)
  OneShot's record (event timeline):
  act_096200983a97  charge cus_ada 42.00 USD  SUCCEEDED
    +  0.00s (new) -> PENDING  accepted and durably recorded
    +  0.07s PENDING -> IN_FLIGHT  attempt 1: claimed by worker-87193, intent recorded before
sending
    +  0.07s IN_FLIGHT -> SUCCEEDED  provider confirmed: applied as eff_00001
  act_e1e0de824cfe  charge cus_ada 42.00 USD  SUCCEEDED
    +  0.00s (new) -> PENDING  accepted and durably recorded
    +  0.07s PENDING -> IN_FLIGHT  attempt 1: claimed by worker-87193, intent recorded before
sending
    +  0.07s IN_FLIGHT -> SUCCEEDED  provider confirmed: applied as eff_00002
  Provider's side of the story (its log):
    POST /v1/charges ref=act_096200983a97 key=act_096200983a97 mode=success -> APPLIED eff_00001,
responding 201
    POST /v1/charges ref=act_e1e0de824cfe key=act_e1e0de824cfe mode=success -> APPLIED eff_00002,
responding 201
  Provider's independent ledger:
    act_096200983a97: 1 effect(s) ['eff_00001']
    act_e1e0de824cfe: 1 effect(s) ['eff_00002']
  > run reconciliation against the provider's ledger
Reconciliation (applied): 2 actions vs 2 provider effects
┌──────────────────────────┬──────────────────┬────────────────┬──────────────────────────┐
│ finding                  │ action           │ provider effects │ details                │
├──────────────────────────┼──────────────────┼────────────────┼──────────────────────────┤
│ SUSPECTED_DUPLICATE_REQUEST │ act_e1e0de824cfe │ -            │ act_e1e0de824cfe has the │
│                          │                  │                │ same type and payload as │
│                          │                  │                │ act_096200983a97, submitted │
│                          │                  │                │ 0.0s apart under different │
│                          │                  │                │ Idempotency-Keys         │
└──────────────────────────┴──────────────────┴────────────────┴──────────────────────────┘
                    2 action(s) matched the provider exactly (hidden; use --all)
  [PASS] reconciliation flags the fresh-key duplicate
  Lesson: Same Idempotency-Key => OneShot returns the original action (no new work). A NEW key for
the same intent looks like a new request: OneShot executes both, and reconciliation can only WARN
about it.

Scorecard (every expectation checked against
the provider's real ledger)
┌──────────────────┬────────┬──────┬────────┐
│ scenario         │ checks │ time │ result │
├──────────────────┼────────┼──────┼────────┤
│ agent-duplicates │    2/2 │ 0.2s │ PASS   │
└──────────────────┴────────┴──────┴────────┘
```

Observed: the slow-provider gap (demo)

```
$ python -m oneshot demo slow-provider-duplicate
# captured 2026-11-03 by scripts/capture_outputs.py (exit code 0)

──────────────────────────────────────────── OneShot demo ────────────────────────────────────────────
Each scenario uses a fresh provider process with its own ledger, a fresh OneShot database, and real
worker processes. Faults are injected at the provider (or as failpoints in the worker).
Working directory: $TMPDIR/oneshot-demo-osurji3i

1/1  slow-provider-duplicate: HONEST GAP: provider is slower than OneShot's settle assumption ──────
  > fault armed at the provider: slow_success on /v1/messages
  > agent POST /actions Idempotency-Key=agent-notify-2004 -> HTTP 201 act_fedb71df991d: message to
+15550100: 'Your refund is on its way'
  > start worker process
  > worker process finished: every action is SUCCEEDED, FAILED or UNKNOWN (exit 0)
  ...waiting for the slow original request to land at the provider
  OneShot's record (event timeline):
  act_fedb71df991d  message to +15550100: 'Your refund is on its way'  SUCCEEDED
    +  0.00s (new) -> PENDING  accepted and durably recorded
    +  0.07s PENDING -> IN_FLIGHT  attempt 1: claimed by worker-87207, intent recorded before
sending
    +  1.07s IN_FLIGHT -> VERIFYING  timed out waiting for the response; outcome unknown and
provider does not deduplicate -> check provider's record at settle time (+0.50s) before deciding
    +  1.62s VERIFYING -> PENDING  no record 1.5s after the last send -> treated as NOT applied
```

```
(assumes provider settles within that window) -> safe to retry
    +  1.63s PENDING -> IN_FLIGHT  attempt 2: claimed by worker-87207, intent recorded before
sending
    +  1.63s IN_FLIGHT -> SUCCEEDED  provider confirmed: applied as eff_00001
  Provider's side of the story (its log):
    POST /v1/messages ref=act_fedb71df991d key=- mode=slow_success -> processing slowly (4s) before
applying
    POST /v1/messages ref=act_fedb71df991d key=- mode=success -> APPLIED eff_00001, responding 201
    POST /v1/messages ref=act_fedb71df991d key=- mode=slow_success -> APPLIED eff_00002, responding
201
  Provider's independent ledger:
    act_fedb71df991d: 2 effect(s) ['eff_00001', 'eff_00002']
  [PASS] the duplicate really happened (2 effects)
  > run reconciliation against the provider's ledger
Reconciliation (applied): 1 actions vs 2 provider effects
```

finding	action	provider effects	details
DUPLICATE_EFFECT	act_fedb71df991d	eff_00001, eff_00002	provider applied this action 2 times (eff_00001, eff_00002); duplicate slipped past every layer - compensate (e.g. refund) and investigate

```
                  0 action(s) matched the provider exactly (hidden; use --all)
  [PASS] reconciliation flags DUPLICATE_EFFECT
  Lesson: Verify-then-retry assumes the provider finishes within the settle window (1.5s here). This
one takes 4s: the lookup finds nothing, OneShot resends, then the original lands. TWO messages.
Without provider-side deduplication a client must choose: retry after verifying (risking this
duplicate) or never retry an ambiguous send (risking a lost message). Reconciliation DETECTS the
duplicate; it cannot undo it.

Scorecard (every expectation checked against the
provider's real ledger)
```

scenario	checks	time	result
slow-provider-duplicate	2/2	4.3s	PASS

Protects against: agent retries with the same key (including concurrent ones); two workers running one action; a stalled worker overwriting a newer result; duplicate charges from OneShot's own retries on idempotent endpoints; blind resends on non-idempotent endpoints.

Cannot protect against: the five gaps above. It can **detect** gaps 1, 2, 3 and 5 by reconciliation, and gap 4 too when it happens. Detection is not undo: refunds or apologies are a business process outside OneShot.

Design decisions and alternatives

- **The provider key is the action id, not the agent's key.** The agent's key is scoped to the agent's API; reusing it upstream would leak a client-controlled value into a third-party namespace and couple two contracts. The action id is unique, stable and ours.
- **Rejected: content-based deduplication as a hard block.** "Same customer, same amount within 5 minutes" also blocks legitimate repeat purchases. OneShot warns instead.
- **Rejected: at-least-once retries for non-idempotent endpoints.** That is simpler, but it guarantees duplicates on every lost response. Verify-then-retry narrows the window to "provider slower than the settle time".
- **Possible alternative: at-most-once for non-idempotent endpoints** (never resend after ambiguity; go straight to UNKNOWN). It removes gap 2 entirely at the cost of messages that are never sent. That is a legitimate product choice; with the current design it is a one-line policy change for a given action type.

Self-check questions

1. An agent's request to OneShot times out and the agent retries with the same key. How many actions exist, and what status code does the retry get?
2. Why can't OneShot simply refuse a request whose payload equals one submitted a minute ago?
3. What makes the slow-provider duplicate impossible for `payment.charge`?
4. Which gap would disappear if `message.send` used at-most-once semantics, and what would be traded for it?
5. Why is the provider idempotency key stable across crashes?

Answers. (1) One action; 200 with `Idempotent-Replayed: true` (or 201 if the first request never reached OneShot). (2) It may be a legitimate repeat, such as two identical orders. Only a human or the agent knows. (3) The provider honours keys: the same-key retry gets 409 while the original runs and a replay afterwards, never a second charge. (4) Gap 2; the trade is that a lost response may mean a message that is never sent. (5) It is the action id, generated once at submission and persisted before any attempt.